

SISTEMAS OPERACIONAIS

UNIDADE DIDÁTICA I

Introdução aos Sistemas Operacionais
e Conceitos de Hardware

341 Conceitos & História

342 Tipos de SO

343 Memória

344 E/S & Barramento

345 Tradutor & Loader

```
$ init --boot
Loading kernel...
Mounting /proc
Starting services
OS ready ✓
```

341

Conceitos iniciais, História e Função dos Sistemas Operacionais

O que é um SO? Para que serve? Como surgiu?

O que é um Sistema Operacional?

341

- Software que gerencia o hardware do computador
- Interface entre usuário/aplicativos e o hardware
- Gerencia recursos: CPU, memória, disco, E/S
- Fornece serviços para programas rodarem
- Abstrai complexidade do hardware

Analogia do dia a dia:

- Pense no SO como o gerente de um hotel.
- Os hóspedes (programas) fazem pedidos e o gerente (SO) aloca quartos (memória), organiza limpeza (CPU), atende ligações (E/S) — tudo sem que o hóspede precise saber como funciona.

Exemplos de SOs:

- Windows, macOS, Linux, Android, iOS

1ª Geração (1945-55)

Sem SO. Programação em cartões perfurados ou painéis. Uma tarefa por vez, manualmente.

2ª Geração (1955-65)

Surgem os sistemas em lote (batch). Operadores agrupavam jobs semelhantes. Mainframes com IBM 7094.

3ª Geração (1965-80)

Multiprogramação e time-sharing. IBM 360, Unix (1969). Múltiplos usuários simultâneos.

4ª Geração (1980-hoje)

PCs pessoais: MS-DOS, Windows, macOS. Internet, redes, smartphones e nuvem.

- Gerência de processos (criação, escalonamento, término)
- Gerência de memória (alocação e proteção)
- Gerência de arquivos e sistema de arquivos
- Gerência de dispositivos de E/S
- Segurança e controle de acesso
- Interface com o usuário (GUI ou CLI)

Comparando com o cotidiano:

- Abrir o Spotify enquanto baixa um arquivo e navega no Chrome — o SO gerencia a CPU entre esses 3 processos, sem que você perceba a troca.

Modo Kernel vs Modo Usuário:

- O SO roda em modo privilegiado (kernel) com acesso total ao hardware. Seus apps rodam em modo usuário, com acesso restrito e seguro.

342

Tipos de Sistemas Operacionais

Monoprogramáveis, Multiprogramáveis e com Múltiplos Processadores

Definição

Executa um único programa por vez. Toda a memória e CPU disponíveis para aquele único processo.

Problema: Ociosidade

Enquanto o programa aguarda leitura de disco (lenta), a CPU fica parada — DESPERDÍCIO!

Exemplo Histórico

MS-DOS: ao rodar um programa, o usuário não podia fazer outra coisa. Semáforo vermelho permanente.

Analogia Moderna

Como um caixa de banco atendendo 1 cliente de cada vez, fechando a fila para os demais.

Definição

Vários programas em memória ao mesmo tempo, compartilhando CPU. Enquanto um espera E/S, outro usa a CPU.

Batch (Lote)

Jobs executados em sequência automática, sem interação do usuário. Usado em mainframes e servidores de banco.

Time-Sharing (Tempo Compartilhado)

CPU dividida em fatias de tempo (time slices). Cada usuário/processo recebe sua fatia. Base do Linux, Windows e macOS.

Tempo Real

Resposta garantida em tempo determinístico. Usado em marcapassos, freios ABS, controle de fábricas.

- SMP (Multiprocessamento Simétrico)
- Todos os processadores compartilham memória e SO
- NUMA: acesso não-uniforme à memória
- Multicore: vários núcleos no mesmo chip
- Cluster: múltiplas máquinas trabalhando juntas

Dia a dia — seu smartphone:

- Um Snapdragon com 8 núcleos divide tarefas: núcleos grandes fazem processamento pesado (jogo/vídeo), núcleos pequenos tratam notificações em background — economizando bateria.

Desafio principal:

- Sincronização e condição de corrida — dois processos tentando modificar o mesmo dado ao mesmo tempo pode causar erros. O SO precisa garantir consistência.

343

Conceitos de Hardware, Software e Memória

Memória Principal, Secundária e Cache

- Hardware: componentes físicos do computador
- CPU, RAM, HD/SSD, placa-mãe, teclado, monitor
- Software: programas e dados — intangível
- Software de sistema: SO, drivers, utilitários
- Software de aplicação: navegador, jogo, Word
- Firmware: software gravado no hardware (BIOS/UEFI)

Analogia:

- Hardware é o corpo humano (estrutura física). Software é a mente e os pensamentos (instruções e dados). O SO é o sistema nervoso — conecta tudo.

Camadas do sistema:

- Aplicativos → SO → Drivers → Hardware
- Cada camada abstrai a complexidade da camada abaixo.

Registradores

Memória mais rápida e cara. Dentro do processador. Ex: acumula resultados durante uma soma. Capacidade: bytes.

Cache (L1/L2/L3)

Memória SRAM ultrarrápida entre CPU e RAM. Guarda dados usados com frequência. Tamanho: KB a MB.

RAM (Memória Principal)

Memória de trabalho. Volátil — perde dados ao desligar. Ex: 8/16 GB no seu PC. Velocidade: nanossegundos.

Armazenamento Secundário

HDD e SSD: persistente. Guarda sistema, arquivos e programas. Milissegundos de acesso. Terabytes de capacidade.

- Princípio da Localidade Temporal: dados usados recentemente tendem a ser usados de novo
- Princípio da Localidade Espacial: dados próximos ao que foi acessado serão acessados
- Cache Hit: dado encontrado no cache → rápido!
- Cache Miss: dado não encontrado → busca na RAM
- $L1 < L2 < L3$ em tamanho, mas $L1 > L2 > L3$ em velocidade

Analogia com escritório:

- Documentos na mesa (L1 cache) são acessados em segundos. Na gaveta (L2) em 10s. No armário da sala (L3) em 30s. No arquivo do térreo (RAM) em 1 min. No depósito externo (disco) em 1 hora.

Impacto real:

- Um cache miss pode fazer a CPU esperar 200x mais. Por isso algoritmos que acessam memória sequencialmente (array) são muito mais rápidos do que acesso aleatório (linked list).

344

Dispositivos de E/S, Barramento e Pipelining

Como a informação flui dentro do computador

- Entrada: teclado, mouse, microfone, câmera, scanner
- Saída: monitor, impressora, caixas de som, projetor
- Entrada/Saída: pendrive, HD externo, tela touch, modem
- Cada dispositivo tem um DRIVER no SO
- Driver: tradutor entre o SO e o hardware específico
- Interrupções: dispositivo avisa a CPU quando terminou

Analogia do garçom:

- A CPU é o chef. Os dispositivos de E/S são as mesas de clientes. O driver é o garçom que sabe como se comunicar com cada mesa. Sem o garçom certo, o chef não entende o pedido.

DMA (Direct Memory Access):

- Dispositivos rápidos (disco, rede) transferem dados direto para a RAM sem passar pela CPU — como um entregador que vai diretamente ao almoxarifado, sem interromper o chef.



O que é?

Conjunto de fios/trilhas que conectam os componentes do PC. Transfere dados, endereços e sinais de controle entre CPU, RAM e dispositivos.



Barramento de Dados

Transporta os dados em si. Largura de 8 a 64 bits. Quanto mais largo, mais dados por ciclo. Ex: PCIe x16 move 16 GB/s.



Barramento de Endereços

Indica qual posição de memória será acessada. 32 bits = 4 GB endereçáveis. 64 bits = praticamente ilimitado.



Barramento de Controle

Sinais de leitura/escrita, interrupções, clock. Coordena quem usa o barramento e quando — evita colisões.

- Técnica de divisão da execução de instruções em estágios
- 5 estágios clássicos:
 1. Busca (Fetch): lê instrução da memória
 2. Decodifica (Decode): interpreta a instrução
 3. Executa (Execute): realiza a operação
 4. Acessa memória (Memory)
 5. Escreve resultado (Write-back)

Analogia da fábrica de carros:

- Sem pipeline: um carro completo antes de começar o próximo. Com pipeline: enquanto carro 3 está sendo pintado, carro 4 recebe motor, carro 5 recebe chassi — tudo simultâneo!

Hazards (Perigos):

- Data hazard: instrução 2 precisa do resultado da instrução 1 (ainda não pronto). Control hazard: desvio condicional (if) invalida estágios preenchidos. O SO e compilador ajudam a mitigar esses problemas.

345

Tradutor, Interpretador, Linker, Loader e Depurador

Da ideia do programador ao programa em execução



Compilador (Tradutor)

Converte TODO o código-fonte (C, C++) em código de máquina antes da execução. Gera arquivo .exe ou .o. Mais rápido na execução.



Interpretador

Traduz e executa o código LINHA POR LINHA em tempo real. Python, JavaScript, PHP. Mais flexível, geralmente mais lento.



Linker (Ligador)

Une vários arquivos objeto (.o) e bibliotecas em um único executável. Resolve referências entre módulos separados.



Loader (Carregador)

Carrega o executável do disco para a memória RAM e inicia a execução. Ajusta endereços de memória (relocação).

- Depurador: ferramenta para encontrar e corrigir erros
- Permite pausar execução em pontos específicos (breakpoints)
- Inspecionar valores de variáveis em tempo real
- Executar passo a passo (step-by-step)
- Exemplos: GDB (Linux), Visual Studio Debugger, Chrome DevTools

Fluxo completo — Exemplo em C:

- 1. Você escreve hello.c (código-fonte)
- 2. gcc compila → hello.o (objeto)
- 3. Linker une hello.o + libc.a → hello (executável)
- 4. Você digita ./hello no terminal
- 5. Loader copia para a RAM → execução!

JIT (Just-In-Time):

- Java e C# compilam para bytecode intermediário. Na execução, a JVM/CLR compila partes críticas para código nativo — híbrido compilado+interpretado.

Resumo — Unidade Didática I

341

Conceitos & História

342

Tipos de SO

343

Hardware & Memória

344

E/S & Barramento

345

Tradutor & Loader

Próximo passo: Atividade Gamificada! 